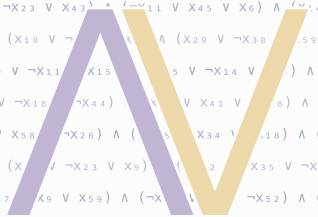
# Graph Attention in a Learned Branching Heuristic for SAT

## *GAT-Q-SAT against Graph-Q-SAT, at matched training set and with two controls*

Donato Meoli

Università di Pisa



# The branching heuristic of a CDCL solver

- A CDCL solver repeatedly decides the value of a variable, propagates, and on a conflict learns a clause and backjumps; the choice of the decision is left to a heuristic, usually VSIDS, which ranks the variables by activity counters.
- The counters carry little information at the start of the search, and a poor decision costs time but never correctness: the heuristic is a natural place for a learned policy.
- Graph-Q-SAT (Kurin et al., 2020) learns it by Deep Q-Learning on the bipartite graph of the formula, and sums the messages of all the neighbors of a node with the same weight.
- This work adds graph attention to its core block (GAT-Q-SAT), and asks where weighting the neighbors helps, and whether the gain comes from the attention or from the added parameters.

# The method

- Branching as a decision process
- Graph-Q-SAT
- GAT-Q-SAT
- The two controls

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*
Veličković et al., *Graph Attention Networks.*

# Branching as a decision process

- The state is the formula simplified by the partial assignment, with the learned clauses; an action is an unassigned variable with a truth value; a step is one decision with the propagation and the conflict analysis it triggers.
- The reward is $-0.1$ at each step and $0$ at termination, so that maximizing the return means terminating in as few decisions as possible.
- DQN approximates $Q^\star(s,a) = \mathbb{E}\big[r + \gamma \max_{a'} Q^\star(s',a')\big]$ with a target network and a uniform replay buffer, and the heuristic is $\pi(s) = \arg\max_a Q_\theta(s,a)$.
- The number of actions changes along an episode and the state is defined up to a permutation of variables and clauses: hence a graph network, which scores each variable node and applies to formulas of any size.

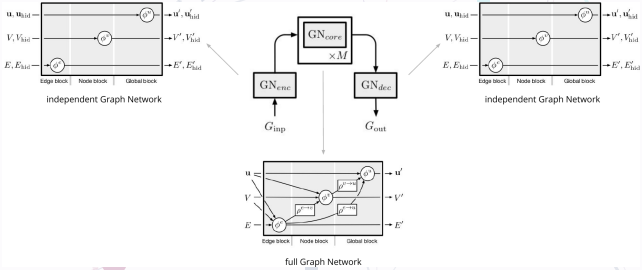
Configuration of Kurin et al.: Adam at $2 \cdot 10^{-5}$, $\gamma = 0.99$, buffer of 20000, minibatches of 64, 50000 updates, at most 500 decisions of the model per episode (the cap).

# Graph-Q-SAT

Nodes: the free variables and the clauses not yet satisfied; edges: the occurrences, with the polarity. A core block, applied $M = 4$ times with shared weights, updates the edges, the nodes and the global attribute:

$$e'_{ij} = \phi^e(e_{ij}, v_i, v_j, u)$$
$$v'_i = \phi^v(\sum_{j \in \mathcal{N}_i} e'_{ij}, v_i, u)$$
$$u' = \phi^u(\bar{e}', \bar{v}', u)$$

The sum gives the same weight to all the clauses of a variable; 71144 parameters.




encoder, core block $\times M$, decoder (after Battaglia et al.)

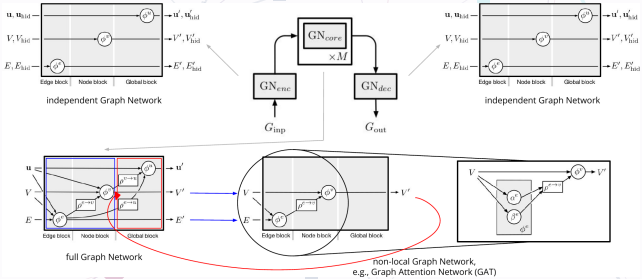
# GAT-Q-SAT

Two edge-featured attention layers between the node and the global update (3 heads, then 1):

$$v_i^+ = \Big\|_k \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k W_k v_j + b$$

$$\alpha_{ij}^k \propto \exp \mathrm{LeakyReLU}(\mathbf{w}_k^\top [W_k v_i \,\|\, W_k v_j \,\|\, W_k^e e'_{ji}])$$

The coefficients depend on the current edge embeddings, which carry the polarity and what the block has added to it. 113256 parameters, about 60% more.




the core block with a non-local block inserted

# Attention or capacity: the two controls

GAT-Q-SAT adds the attention to the sum, so a gain may be due to the attention or simply to the larger network. Two more models, with the same configuration:

- Graph-Q-SAT wide: hidden layers of 104 units in place of 64, the capacity of GAT-Q-SAT with no attention;
- attention aggregation: the sum replaced by a mean weighted by attention (3 heads, averaged), at the capacity of Graph-Q-SAT.

| | Graph-Q-SAT | Graph-Q-SAT wide | attention aggregation | GAT-Q-SAT |
|---|---|---|---|---|
| parameters | 71144 | 113504 | 71627 | 113256 |
| training seeds | 3 | 2 | 2 | 3 |

- The port to the current stack
- The measure and the reference solver

## Port and setup

# The port, and a test of what it computes

- The code of 2020 is written for PyTorch 1.1 with pinned compiled extensions (`torch-scatter`, `torch-sparse`), which can no longer be installed: I ported it to the current PyTorch and PyTorch Geometric.
- The aggregations come from `torch_geometric.utils.scatter`; the environment runs with `gym` and `gymnasium`; the MiniSat wrapper is rebuilt against `numpy` 2; the checkpoints of 2021 load with the current `GATConv`.
- The checkpoints of 2021, evaluated on the new stack, give the same iterations as in 2021 on every instance in 63 of 86 evaluations, and medians within $0.03$ in the others (on `flat30-60`: $1.833$ and $2.000$ on both stacks).
- The training path was checked by retraining the two coloring models from scratch (the seeds of 2026).

# The measure and the reference solver

$$\mathrm{MRIR} = \mathrm{median}\ \frac{\#\text{iterations of MiniSat}}{\#\text{iterations of the model}}$$

over the instances of a family; an iteration is a decision, of the model or of VSIDS, which takes over after the cap ($500$ unless stated otherwise). Above $1$ the model takes fewer decisions than MiniSat.

- The model drives MiniSat without restarts; the denominator is the stronger MiniSat on each family: with restarts on coloring (up to 26% fewer iterations), without on uniform random 3-SAT (2.2 to 2.9 times fewer at 250 variables).
- 100 test instances per family; graph coloring from 30 to 200 nodes, trained on 50.

- Graph coloring, by size
- Capacity or attention
- A change of the training distribution
- Cross-domain transfer
- Uniform random 3-SAT
- Iterations and running time

# Results

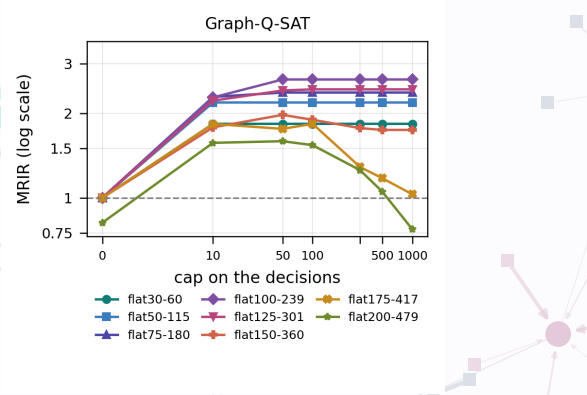
# Graph coloring: the gap grows with the size

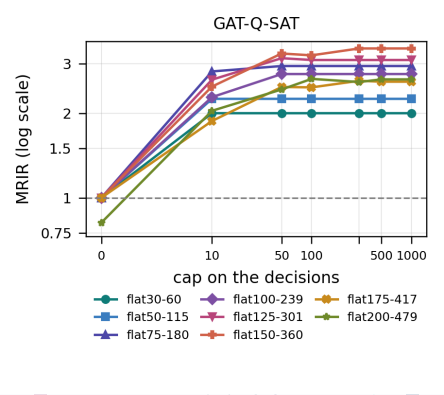
| family | Graph-Q-SAT | GAT-Q-SAT | $\Delta$ |
|---|---|---|---|
| flat30-60 | 1.83 | 2.00 | $+0.17$ |
| flat50-115 | 2.18 | 2.25 | $+0.07$ |
| flat75-180 | 2.37 | 2.94 | $+0.58$ |
| flat100-239 | 2.64 | 2.76 | $+0.12$ |
| flat125-301 | 2.43 | 3.09 | $+0.66$ |
| flat150-360 | 1.74 | 3.40 | $+1.66$ |
| flat175-417 | 1.17 | 2.59 | $+1.42$ |
| flat200-479 | 1.05 | 2.64 | $+1.59$ |

MRIR at cap 500, mean over the seeds

- GAT-Q-SAT takes fewer decisions on all eight families.
- Up to 100 nodes the two are close; from 125 nodes on the gap is large, mostly because Graph-Q-SAT degrades almost to the level of MiniSat at 200 nodes, four times the training size.

# Graph coloring: the MRIR against the cap



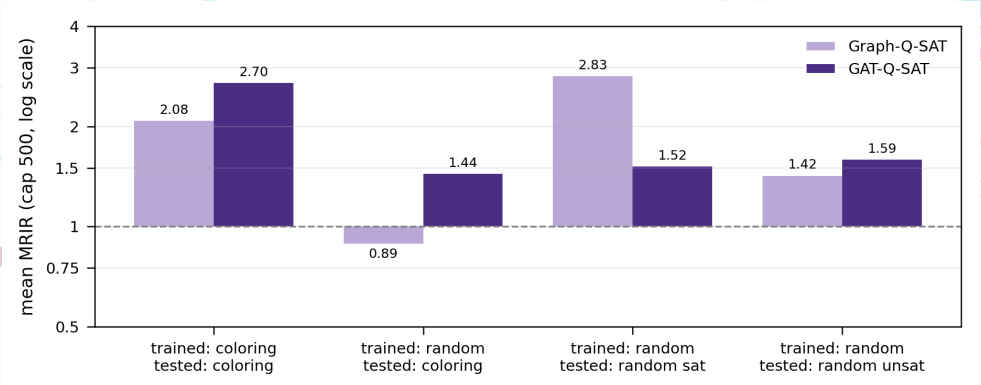




Runs of 2021, one line per family, each in its color (the same in every figure); the point at cap 0 is MiniSat without restarts. On the large families the decisions of Graph-Q-SAT after the first few tens make the search longer.

# Capacity or attention

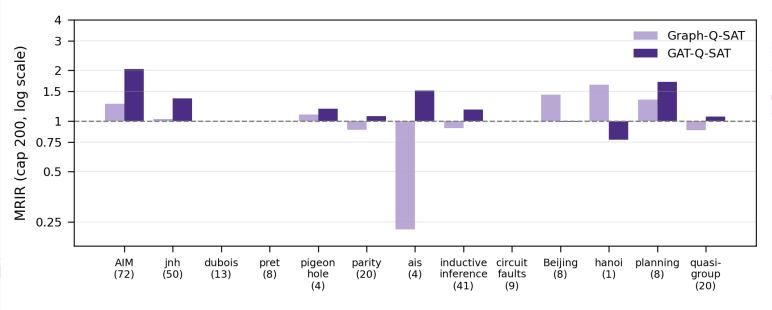
[The four variants on the eight families, from Table 2 of the note: whether Graph-Q-SAT wide closes the gap, whether attention aggregation reaches GAT-Q-SAT at a sixth of its added parameters, and the size of the differences against the variation across the seeds.]

# Trained on random formulas, tested on graph coloring



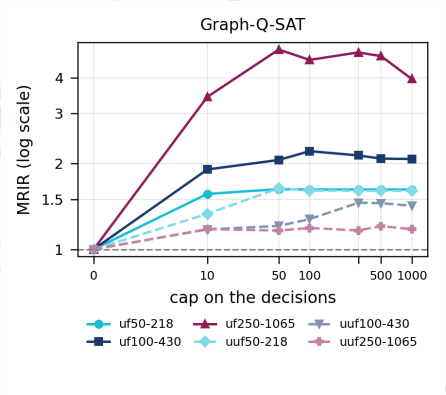

Light lilac Graph-Q-SAT, deep violet GAT-Q-SAT. Trained on uniform random 3-SAT, Graph-Q-SAT is below MiniSat on five of the eight coloring families, GAT-Q-SAT above it on seven: the weighting of the neighbors helps also on a structure not seen in training.

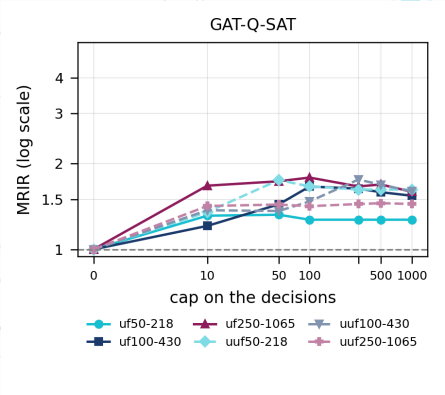
# Cross-domain transfer: thirteen SATLIB families




The coloring-trained models with no retraining, cap 200, MRIR against the stronger MiniSat. On the eleven informative families GAT-Q-SAT is above MiniSat on eight and Graph-Q-SAT on six; GAT-Q-SAT is the better of the two on eight, Graph-Q-SAT on two. Yet, the seeds overlap on all the families but `jnh`: an indication, not a proof.

# Uniform random 3-SAT: the ranking is reversed




On satisfiable formulas Graph-Q-SAT is better at all sizes ($4.78$ against $1.69$ at 250 variables): at the phase transition the neighbors of a node are statistically alike. On unsatisfiable formulas the two are level or GAT-Q-SAT is ahead.

# Iterations and running time

| | cap 50 | | cap 500 | | cap 1000 | |
|---|---|---|---|---|---|---|
| | sec | MRIR | sec | MRIR | sec | MRIR |
| Graph-Q-SAT | 2.55 | 1.59 | 26.51 | 1.05 | 57.67 | 0.77 |
| GAT-Q-SAT | 3.52 | 2.43 | 12.20 | 2.64 | 12.19 | 2.64 |

`flat200-479`, median seconds per instance

- A decision of GAT-Q-SAT costs more, but at larger caps it is the faster of the two, since it terminates in fewer decisions.
- Both take seconds where MiniSat takes a fraction of a second: at this size neither is competitive in time, which is the motivation of the restricted heuristics (implemented, not measured) and of offline inference.

# Conclusions and limits

- On graph coloring GAT-Q-SAT took fewer decisions on every family, with a gap that grows with the size; it was also the better of the two when trained on random formulas and tested on coloring.
- [The controls: attention or capacity.]
- On satisfiable uniform random 3-SAT Graph-Q-SAT was clearly superior, where there is no structure for the attention to weight.
- Limits: three seeds per model (two per control) do not resolve differences of a few tenths; neither model is competitive with MiniSat in time; the largest SATLIB instances (e.g., bounded model checking) are beyond the size I could evaluate.

Code, instances and logs: github.com/dmeoli/GQSAT.

# Thank you for your *attention*!

github.com/dmeoli/GQSAT